

Autonomous Fire Patrol Robot

4-Week Execution Plan · ESP32 + 4WD + MQ-2 + DHT22 + WiFi + Telegram

Week 1
PC-only preparation

Week 2
Sensors

Week 3
Autonomous patrol

Week 4
WiFi alerts

WEEK 1

PC-only preparation & simulation

No hardware needed — code, simulate, and design from anywhere

You're travelling this week, so Week 1 is fully PC-based. You'll set up your development environment, master the ESP32 ecosystem, write and simulate all motor and sensor code, and produce a complete wiring diagram — so the moment you're back home, you plug in and run.

DAILY TASKS — STEP BY STEP

- 1 Install Arduino IDE 2.x and add ESP32 board support via Board Manager (URL: espressif/arduino-esp32)
- 2 Study the ESP32-WROOM-32 pinout: identify digital GPIO, ADC analog pins, PWM-capable pins, 3.3V and GND rails
- 3 Write motor control library from scratch: forward(), backward(), turnLeft(), turnRight(), stop() using L298N IN1-IN4 + PWM on ENA/ENB
- 4 Open Wokwi.com in browser — build virtual circuit: ESP32 + L298N + 4 DC motors, test all movement functions live
- 5 Write DHT22 sensor code using DHT.h library — read temperature and humidity, print to serial every 2 seconds
- 6 Add virtual DHT22 to Wokwi simulation, simulate different temperature values and verify readings
- 7 Write MQ-2 analog read code: read ADC pin, map raw value to ppm approximation, set danger threshold constant
- 8 Simulate MQ-2 in Wokwi using a potentiometer — turn knob past threshold, trigger virtual buzzer
- 9 Draw complete wiring diagram in draw.io (free, browser-based): ESP32, L298N, MQ-2, DHT22, buzzer, power rails with correct colours
- 10 Create GitHub repo, push all code with clear folder structure: /motor, /sensors, /diagrams

COMPONENTS & TOOLS

Arduino IDE 2.x (free)	Wokwi.com (browser sim)	draw.io (browser)	ESP32 datasheet PDF
L298N datasheet	DHT.h library	GitHub account	VS Code (optional)

SKILLS YOU LEARN

Arduino IDE setup	ESP32 GPIO architecture	Wokwi circuit simulation	PWM motor control theory
ADC sensor reading	Wiring diagram design	Version control (Git)	Code modularisation

MILESTONE

All motor + sensor code written and tested in Wokwi simulation. Full wiring diagram complete and pushed to GitHub.

Sensors, detection & alarm

Wire real sensors, calibrate thresholds, trigger the buzzer

Back home with hardware. Add the safety brain of the system. By the end of Week 2, your robot detects smoke and abnormal temperature simultaneously before sounding the alarm — dual-sensor validation that eliminates false alerts.

DAILY TASKS — STEP BY STEP

- 1

Wire MQ-2 smoke sensor: VCC to 5V, GND to GND, AOUT to ESP32 ADC pin (GPIO34 or 35)
- 2

Power on and log baseline MQ-2 ADC values to serial monitor in clean air — record the number
- 3

Wave lighter or incense near sensor (do NOT ignite), observe ADC spike — note detection range
- 4

Wire DHT22: VCC to 3.3V, GND to GND, DATA to GPIO with 10kΩ pull-up resistor to 3.3V
- 5

Upload DHT22 code from Week 1, confirm temperature and humidity read correctly on serial monitor
- 6

Write combined threshold logic: DANGER state only if smoke ADC > X AND temperature > Y degrees
- 7

Wire active buzzer to a GPIO pin, write alarm() function with distinct beep patterns for WARN vs DANGER
- 8

Add 3-sample debounce: only enter DANGER if condition holds for 3 consecutive reads 500ms apart
- 9

Test full detection pipeline: smoke near sensor + heat source nearby → buzzer triggers with correct pattern
- 10

Fine-tune X and Y thresholds, document final calibrated values with comments in code

COMPONENTS & TOOLS				SKILLS YOU LEARN			
MQ-2 smoke sensor	DHT22 sensor	Active buzzer	10kΩ resistors (×3)	ADC sensor reading	Pull-up resistor wiring	Dual-sensor fusion	Signal debouncing
Breadboard	Jumper wires (M-F, M-M)	USB cable	Multimeter (helpful)	Threshold calibration	Serial monitor debugging	Alarm pattern design	Hardware troubleshooting

MILESTONE
Robot detects smoke + high temperature simultaneously and triggers a distinct patterned buzzer alarm. Zero false alarms in

Autonomous patrol & obstacle avoidance

Combine motion + detection into a real self-navigating robot

The hardest and most impressive week. Merging movement with real-time sensor reading requires non-blocking code using `millis()` instead of `delay()` — this is the core embedded systems concept that makes this project stand out on any CV or portfolio.

DAILY TASKS — STEP BY STEP

- 1
- Attach 4WD chassis motors to L298N driver — double-check wiring from Week 1 diagram
- 2
- Upload motor code, verify all four directions work cleanly on the physical chassis
- 3
- Mount HC-SR04 ultrasonic sensor at front of chassis: VCC 5V, GND, TRIG and ECHO to GPIO pins
- 4
- Write `getDistance()` function using `pulseIn()` — test by printing distances as you move hand toward sensor
- 5
- Write avoidance sequence: if distance < 20cm → stop → reverse 300ms → turn right 400ms → continue forward
- 6
- Rewrite main loop using `millis()` timer — replace all `delay()` calls, ensure sensor reads happen every 200ms while moving
- 7
- Implement patrol state machine: PATROL → OBSTACLE_AVOID → PATROL, and PATROL → DANGER → HALT
- 8
- In DANGER state: stop motors immediately, sound alarm, hold position until reset button pressed
- 9
- Test in real room with chairs and obstacles — log position zone, smoke, temp to serial every patrol cycle
- 10
- Record a short video of the robot navigating and detecting — useful for your portfolio

COMPONENTS & TOOLS				SKILLS YOU LEARN			
HC-SR04 ultrasonic sensor	4WD chassis (assembled)	L298N motor driver (wired)	7.4V LiPo or 4×AA pack	Autonomous navigation	State machine architecture	Non-blocking <code>millis()</code> loops	Ultrasonic distance sensing
Push button (reset)	Zip ties / mounting tape	Servo SG90 (optional scanner)	Smartphone (for video)	Source localisation	System integration testing	Multi-sensor data fusion	Embedded debugging

MILESTONE

Robot patrols the room autonomously, avoids all obstacles, and halts with alarm when fire conditions detected. Runs 30 min

WiFi alerts, polish & documentation

Networked IoT safety system — portfolio and CV ready

The Telegram notification transforms your project from 'robot that beeps' into 'networked IoT safety system with remote monitoring'. This framing matters enormously for interviews and your portfolio. Finish strong with proper documentation — a well-documented embedded project is a professional one.

DAILY TASKS — STEP BY STEP

- 1
- Add WiFi credentials to code — connect ESP32 using `WiFi.begin()`, verify IP prints to serial on boot
- 2
- Install `UniversalTelegramBot` library via Arduino Library Manager
- 3
- Create Telegram bot: open `@BotFather`, `/newbot`, copy the token. Open `@userinfobot` to get your chat ID
- 4
- Write `sendAlert(zone, smoke_ppm, temp)` function — sends formatted Telegram message on DANGER state
- 5
- Test alert: trigger smoke detection, confirm Telegram message arrives within 3 seconds
- 6
- Add periodic status heartbeat: every 5 minutes send 'All clear — zone X, YY°C, ZZ% humidity'
- 7
- Optional: set up simple ESP32 `AsyncWebServer` dashboard — HTML page shows live sensor values at robot's IP
- 8
- Run 2-hour full autonomous patrol stress test — monitor WiFi stability, memory usage (`ESP.getFreeHeap()`)
- 9
- Write `README.md`: project description, features list, wiring diagram image, setup instructions, photos
- 10
- Final demo: patrol the room, intentionally trigger detection, show Telegram alert arriving on phone

COMPONENTS & TOOLS				SKILLS YOU LEARN			
WiFi network (2.4GHz)	Telegram app	UniversalTelegramBot lib	HTTPClient library	Embedded WiFi networking	REST API HTTP calls	IoT messaging protocols	System reliability testing
ArduinoJson library	GitHub repo	Smartphone for demo	Camera for photos	Memory management	Technical writing	README documentation	Demo presentation

MILESTONE

Full autonomous robot sends real-time Telegram alerts with sensor data. 2-hour stress test passed. GitHub repo with full docs

Tip: each week builds on the last. Week 1 simulation means you already know your code works before touching a single wire. That's the professional embedded systems mindset.